WORKSHOP 研讨/分享/感悟

仓颉社区中的灵感碰撞 / 社区先行者的干货分享

仓颉 Cangjie

三方库Editor介绍

主讲人：赵俊杰

关键词

OHOS Cangjie Editor

FFI tree-sitter



目录

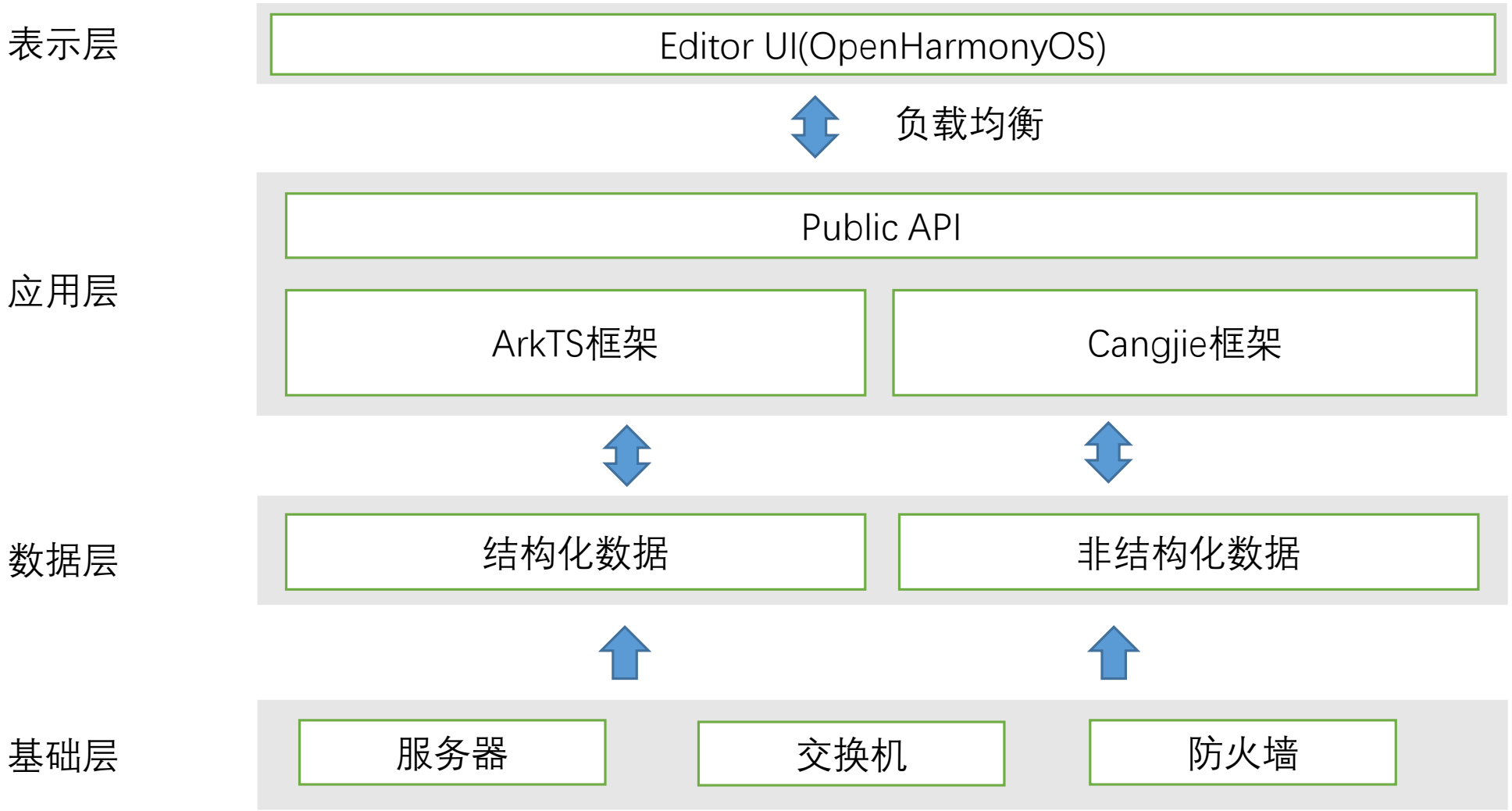
- Editor 背景介绍
- Editor 架构设计
- Editor 功能
- 使用中涉及的仓颉特性
- Editor代码演示
- 未来规划

Editor背景介绍

Editor是多语言代码编辑器库，用户可以通过集成该库，为开发者提供编写代码的界面。为了有良好的编码体验，具备以下核心优势：

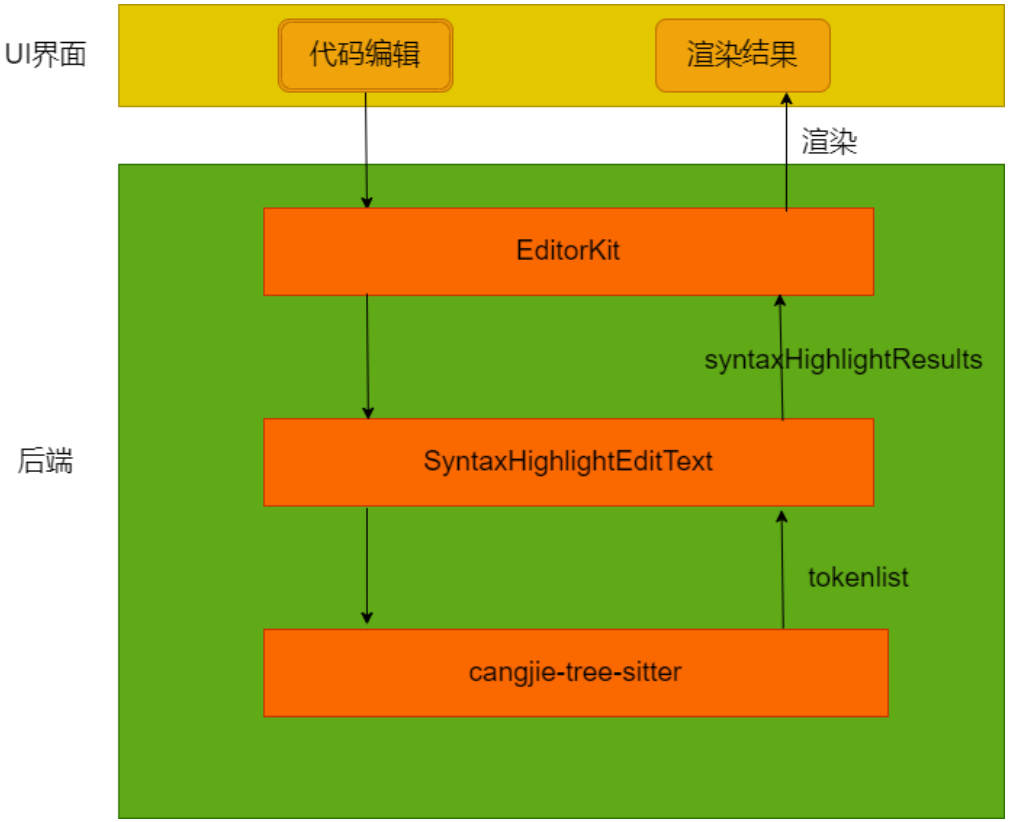
1. 高效并发：轻量级线程和简单易用并发机制以及同步机制
2. 生态兼容：与C（FFI机制）进行互操作
3. 语言扩展性：支持C、Swift、Go、Java、Kotlin语言等多种语言编程
4. 仓颉：支持仓颉语言编程
5. 高性能词法解析：基于Tree-sitter实现

Editor架构设计

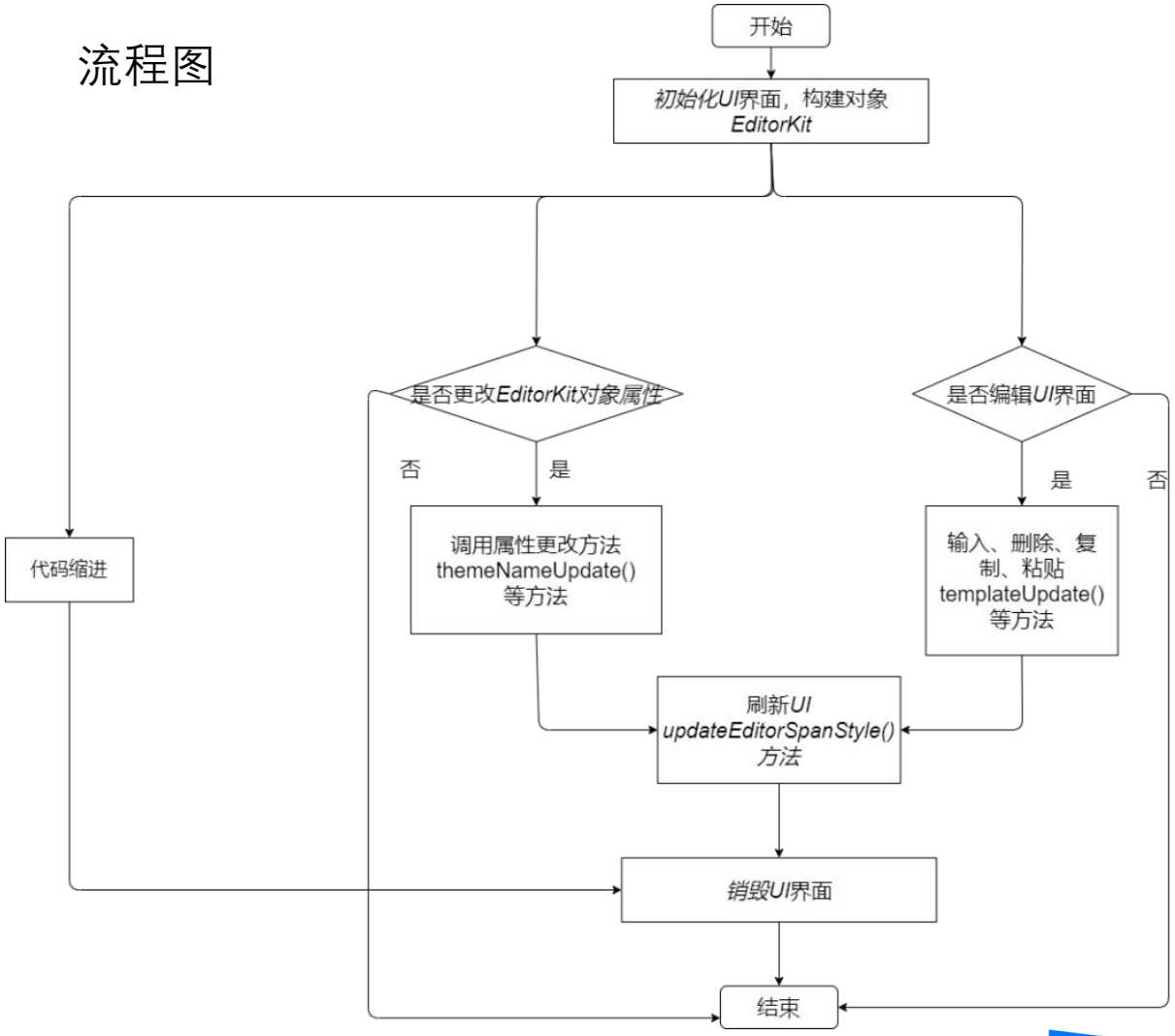

表示层
Editor UI(OpenHarmonyOS)
负载均衡
应用层
Public API
ArkTS框架
Cangjie框架
数据层
结构化数据
非结构化数据
基础层
服务器
交换机
防火墙

应用层架构流程

内部实现细节：



流程图



仓颉 Cangjie

代码编辑器功能

1. 支持代码编辑功能
2. 支持代码高亮功能，当前支持C、Cangjie、Go、Java、Kotlin、Swift、Python、Python3、Rust、JavaScript、TypeScript、C#、C++、PHP、Ruby、SQL、Dart、Scala、Erlang、Elixir、Lua、Racket、Objective-C、Objective-C++、Haskell、Bash、HTML、Perl、R语言（共29种）
3. 支持代码滚动功能
4. 支持行号显示功能
5. 支持自动补全功能
6. 支持自动缩进功能
7. 支持语言+模板设置功能
8. 支持文本设置功能
9. 支持光标处文本插入功能
10. 支持字体大小设置
11. 支持高亮主题切换
12. 支持代码编辑器高度、代码编辑器宽度的设置

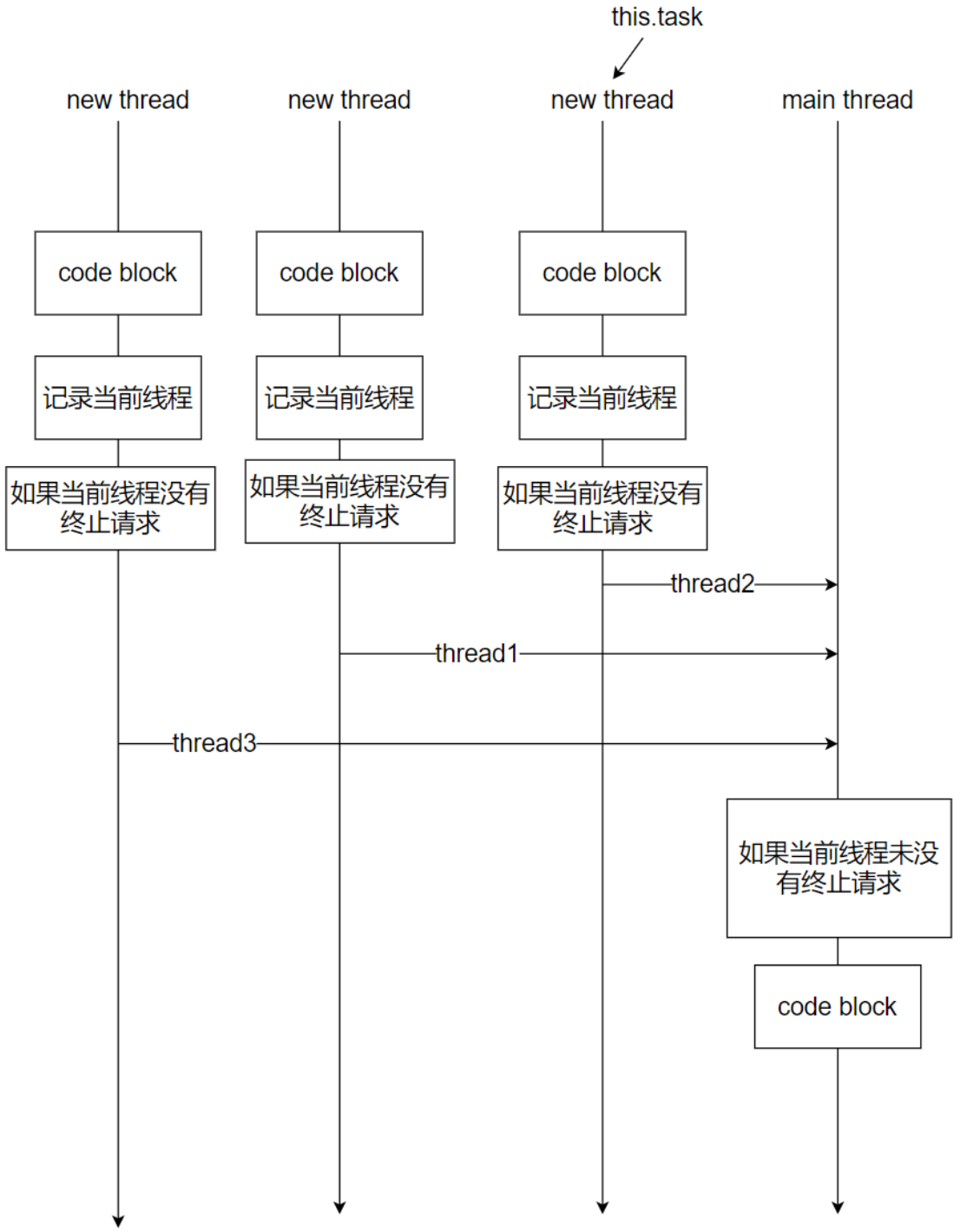
涉及的仓颉特性

1. 仓颉与C语言使用FFI机制交互
2. 轻量并发编程与同步机制
3. 定时器
4. 包管理机制
5. 错误处理

WORKSHOP 研讨/分享/感悟

轻量并发编程

```
private var task: Future<Unit>
let newThread = spawn { =>
    try {...} catch (e: Exception) {
        AppLog.info("syntaxHighlight exception: ${e}")
    }
}
this.task = newThread
this.task.cancel()
```



同步机制：可重入互斥锁 ReentrantMutex

```
this.treeMutex = ReentrantMutex()
if (this.languageName != languageName) {
    this.languageName = languageName
    this.treeMutex.lock()
    if (!this.tree.isNull()) {
        this.tree.setTree(CPointer<Unit>())
    }
    this.treeMutex.unlock()
    this.bufferMutex.lock()
    this.bufferTokenlist.clear()
```

同步机制引入原因：在并发编程中，如果缺少同步机制来保护多个线程共享的变量，很容易会出现数据竞争问题（data race）。仓颉编程语言提供三种常见的同步机制来确保数据的线程安全：原子操作，互斥锁以及条件变量。Editor引入了可重入互斥锁 **ReentrantMutex**

并发线程通信可靠性

Editor通过以下2点保证了并发可靠性：

1. 多线程时，保证子线程资源创建与释放独立可靠。
   如果线程间资源互相依赖会形成死锁，资源的自动创建与释放无法明确指定，但是资源的手动创建与释放必须独立可靠。

2. 多线程时，保证子线程间不存在数据竞争关系。
   通过互斥锁避免数据竞争，例如对有数据竞争关系的数据加可重入互斥锁ReentrantMutex。

Editor代码演示

Editor 字体设置

```
var editorKitController: EditorKitController = EditorKitController(wid

protected override func aboutToAppear() {
    editorKitController.setLanguageAndText(LANGUAGETYPE.C, this.text)
    editorKitController.setText(this.text)
    editorKitController.insertTextOnCursor("")
    editorKitController.setTabLen(4)
    editorKitController.setFontSize(13)
    editorKitController.setTheme(theme)
    editorKitController.setWidth(380.0)
    editorKitController.setHeight(600.0)
```

```
class Solution {
public:
    double findMedianSortedArrays(vector<int>& nums1,
vector<int>& nums2) {

    }
};
```

```
class Solution {
public:
    double findMedianSortedArrays(vector<int>&
nums1, vector<int>& nums2) {

    }
};
```

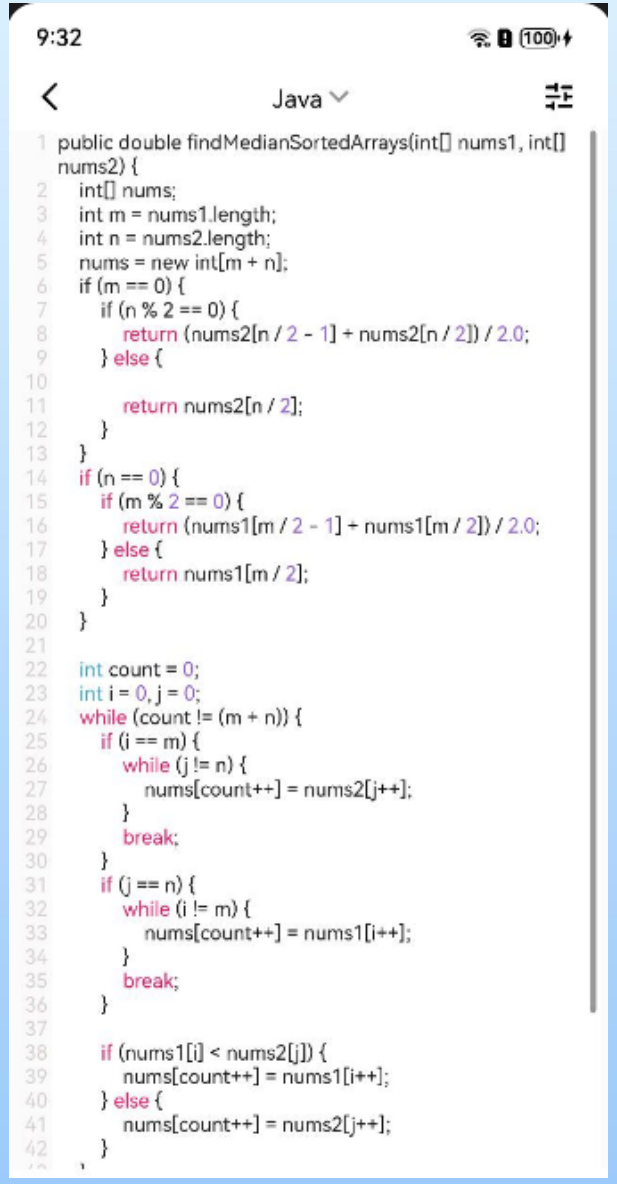
```
class Solution {
public:
    double
findMedianSortedArrays(vector<int>
& nums1, vector<int>& nums2) {

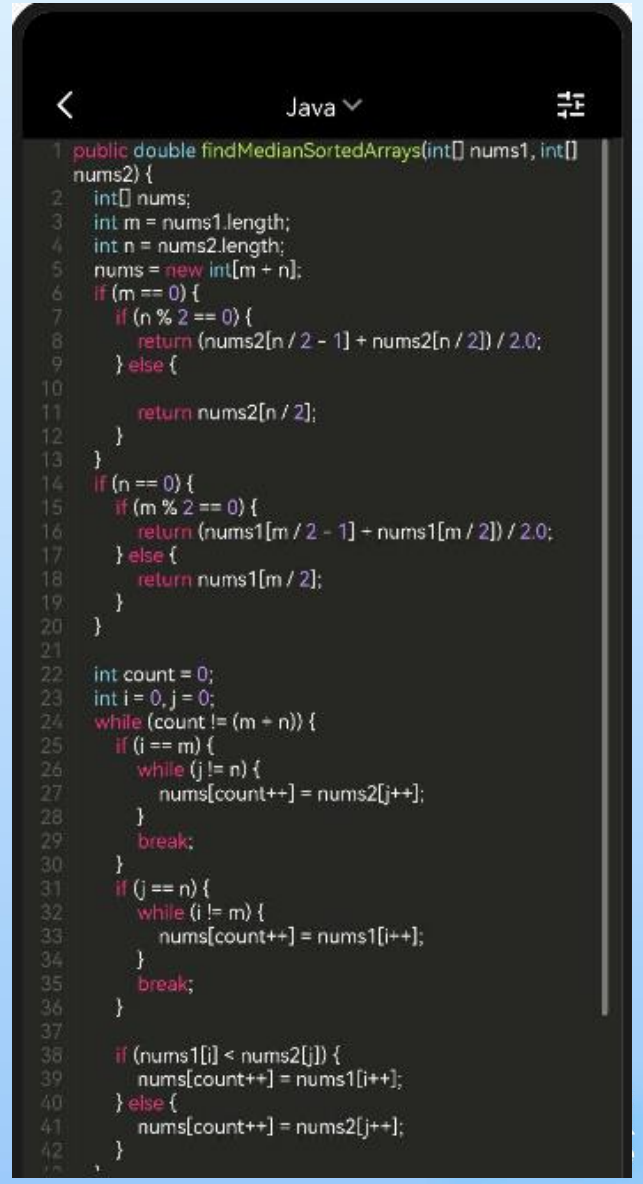
    }
};
```

仓颉 Cangjie

Editor代码演示

Editor 高亮展示 + 主题切换



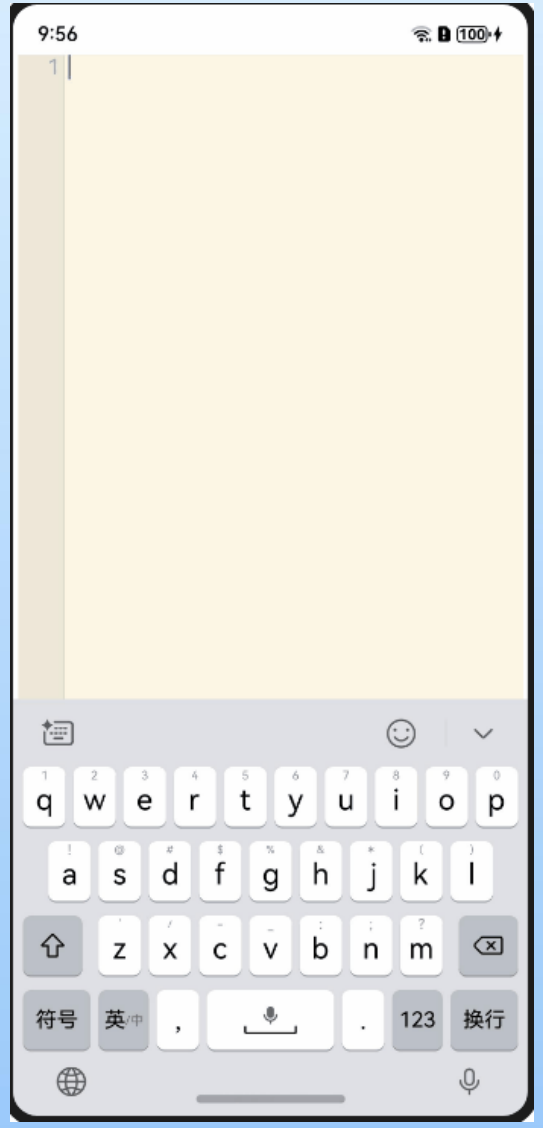


Editor代码演示

Cangjie代码展示

```
from std import random.*

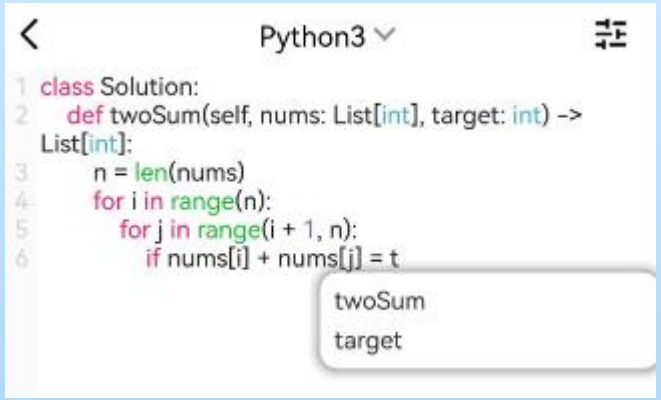
mian() {
    let speed = Random().nextFloat()
    println("${speed} km/s")
    let level = if (speed > 16.7) {
        "第三宇宙速度，鹊桥相会"
    } else if (speed > 11.2) {
        "第二宇宙速度，嫦娥奔月"
    } else if (speed> 7.9) {
        "第一宇宙速度，环游世界"
    } else {
        "脚踏实地，仰望星空"
    }
    println(level)
}
```

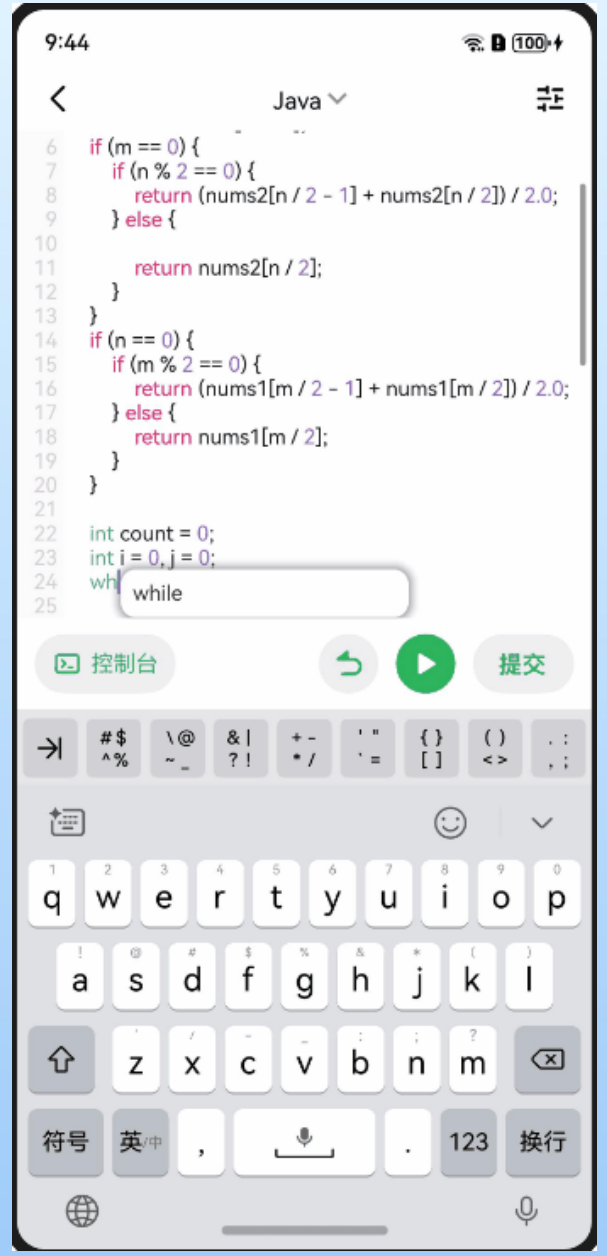


仓颉 Cangjie

Editor代码演示

Editor自动补全功能





未来规划

仓库地址: https://gitcode.com/Cangjie-TPC/editor4cj

Editor 属性的设置：对字体的设置

Editor 功能的多样：手势控制字体的缩放

Editor 性能的提升：提升代码量限制，更流畅的代码展示。

仓颉 Cangjie

工程接入

通过module引入

1. 项目下载，将editor模块拷贝到应用目录下
2. 修改自身应用entry下的 oh-package.json5 文件，在dependencies字段添加”editor”: “file:filepath”

```
{
  "license": "",
  "devDependencies": {},
  "author": "",
  "name": "entry",
  "description": "Please describe the basic information.",
  "main": "",
  "version": "1.0.0",
  "dynamicDependencies": {},
  "dependencies": {
    "editorkit": "file:../third_party/editor4cj/editorkit"
  }
}
```

3. 修改自身应用entry/src/main/cangjie下的cjpm/toml 文件，在[dependencies]字段下添加 editorkit = {path: = “filepath”}

```
[dependencies]
  cj_res_entry = {path = "./cj_res", version = "1.0.0"}
  editorkit = {path = "../../../../third_party/editor4cj/editorkit/src/main/cangjie", version = "1.0.0"}
```

4. 在项目中使用引入项目：

```
internal import editorkit.editor_kit.*
internal import editorkit.editor_kit.utils.*
internal import editorkit.editor_kit.editorKitUtils.*
```

仓颉 Cangjie

WORKSHOP 研讨/分享/感悟

仓颉社区中的灵感碰撞 / 社区先行者的干货分享

仓颉 Cangjie

感谢您的观看!

关注“仓颉编程语言”公众号，
获取最新官方资讯!

